```
... === HAMMING CODE - ERROR DETECTION AND CORRECTION ===

    Enter number of 4-bit data blocks in the stream: 1011

    --- Block 1 ---
    Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
    Original Data Bits             : 1011
    Encoded Hamming Code (7 bits) : 0110011
    Enter bit position to introduce error (1-7, 0 = no error): 3
    Transmitted Code (error at position 3) : 0100011
    Received Code : 0100011
    Checking for errors...
    Error detected at bit position: 3 -> Correcting...
    Corrected Code : 0110011
    Recovered Data Bits : 1011

    --- Block 2 ---
    Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
    Original Data Bits             : 1011
    Encoded Hamming Code (7 bits) : 0110011
    Enter bit position to introduce error (1-7, 0 = no error): 2
    Transmitted Code (error at position 2) : 0010011
    Received Code : 0010011
    Checking for errors...
    Error detected at bit position: 2 -> Correcting...
    Corrected Code : 0110011
    Recovered Data Bits : 1011

    --- Block 3 ---
    Enter 4 data bits (space separated, e.g. 1 0 1 1):
```

EX-NO:- 5

26-08-2026

ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM:-

To write a program to implement error detection and correction using the hamming code.

ALGORITHM:-

1. Start the program.
2. Read the number of 4-bit blocks (n) forming the input data stream.
3. For each 4-bit data block D = d1 d2 d3 d4, perform Hamming encoding.
   a. Arrange 7 bits with parity bits at positions 1, 2 and 4, and data bits at positions 3, 5, 6 and 7.
   b. Compute the parity bits using even parity:
      P1 = d1 XOR d2 XOR d4 (covers bit positions 1, 3, 5, 7)
      P2 = d1 XOR d3 XOR d4 (covers bit positions 2, 3, 6, 7)
      P4 = d2 XOR d3 XOR d4 (covers bit positions 4, 5, 6, 7)
   c. Form the 7-bit hamming code word: P1 P2 d1 P4 d2 d3 d4.
4. Simulate transmission by introducing a single-bit error at a chosen bit position (entered by the user) in the code word.
5. At the receiver, recompute the check bits (syndrome) from the received code word:

C1 = P1 XOR d1 XOR d2 XOR d4

C2 = P2 XOR d1 XOR d3 XOR d4

C3 = P4 XOR d2 XOR d3 XOR d4

6. Compute Error position = C4 C2 C1 (binary number formed from the check bits).

7. If Error Position = 0, no error has occurred.

Else, the bit at "Error Position" is inverted (complemented) to correct the error.

8. Extract the original 4 data bits from the corrected 7-bit code word.

9. Repeat steps 3 to 8 for all n blocks in the data stream.

10. Stop the program.

PROGRAM:-

```
def encode(d):
"""d: list of 4 bits [d1, d2, d3, d4] -> returns 7-bit hamming code
list Positions (1-indexed): 1=P1 2=P2 3=d1 4=P4 5=d2
6=d3 7=d4 """
    h = [0] * 7
    h[2] = d[0]   # Position 3
    h[4] = d[1]   # Position 5
    h[5] = d[2]   # Position 6
    h[6] = d[3]   # Position 7
    h[0] = h[2] ^ h[4] ^ h[6]   # P1 covers 1, 3, 5, 7
    h[1] = h[2] ^ h[5] ^ h[6]   # P2 covers 2, 3, 6, 7
    h[3] = h[4] ^ h[5] ^ h[6]   # P4 covers 4, 5, 6, 7
    return h
```

```
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

error_pos = c1 * 1 + c2 * 2 + c4 * 4 # Syndrome -> faculty bit position
if error_pos != 0:
    print(f" Error detected at bit position : {error_pos} -> correcting...")
    h[error_pos - 1] ^= 1 # flip the faulty bit
else:
    print (" No error detected.")
return error_pos

def to_str (bits):
    return "".join(str(b) for b in bits)

def main():
print ("=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===\n")
n = int(input ("Enter number of 4-bits data blocks in the
Stream : "))
for b in range (1, n+1):
    print(f "\n-- Block {b} ---")
d = list (map(int, input ("Enter 4 data bits (Space separated,
e.g. 1 0 1 1): "), split ()))
    h = encode (d)
    print (f "Original Data Bits : {to_str (d) }")
    print(f"Encoded Hamming Code (7 bits) : {to_str(h)}")
pos = int (input ("Enter bit position to introduce error
(1-7, 0 = no error) : "))
    if 1 <= pos <= 7:
        h[pos - 1] ^= 1
```

Enter number of 4-bit data blocks in the stream : 2

--Block 1---

Enter 4 data bits (Space Separated e.g 1011) : 1011

Original Data Bits : 1011

Encoded Hamming Code (7 bits) : 0110011

Enter bit position to introduce error (1-7, 0-no error) :

Transmitted code (with error induced at pos 3) : 0100011

Received code : 0100011

Checking for errors...

Error detected at bit position : 3 -> correcting ...

corrected code : 0110011

Recovered Data Bits : 1011

---Block 2---

Enter 4 data bits (Space Separated, e.g. 1011) : 0110

Original Data Bits : 0110

Encode Hamming code (7 bits) : 1100110

Enter bit position to introduce error (1-7, 0 = no error) :

Toransmitted code (with error induced at pos 6) : 1100100

Received code : 1100100

Checking for errors....

Error detected at bit position : 6 -> correcting ...

Corrected code : 1100110

Recovered Data Bits : 0110

```
print(f"Transmitted code (with error induced at pos {pos}): {to_
str(h)}")
else:
    print(f"Transmitted code (no error induced): {to_str(h)}")
print(f"Received code : {to_str(h)}")
print("Checking for errors...")
detect_and_correct(h)
print(f"corrected code : {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits : {to_str(rd)}")
if __name__ == "__main__":
    main()
```

RESULT:-

The program to implement error detection and correction using the hamming code concept was successfully written in Python, executed and Verified using a sample input data.